RECOMMENDER SYSTEMS / REINFORCEMENT LEARNING

Movie Bandit Lab

A complete, source-aligned guide to the data pipeline, action space, reward model, four policies, interface, and offline evaluation.

WHAT THIS IS An inspectable contextual multi-armed bandit demonstration. Each policy recommends one of 42 movies, receives a simulated binary response, and can update its choice strategy.

42 movie actions	19 genre features	4 policies	11 viewer profiles

Prepared 27 October 2026
Based on app.py, environment.py, agents.py, run_evaluation.py, README.md, and the saved verification report.

Contents

Contents . . . 2
1. Project purpose and mental model . . . 3
2. Project files and how they fit together . . . 4
3. MovieLens data and preprocessing . . . 5
4. Viewer context and the 11 profiles . . . 6
5. Action space: exactly 42 movie arms . . . 7
5A. Complete ordered candidate slate . . . 8
6. Movie features and popularity . . . 9
7. The environment: probability and reward math . . . 10
8. Policy 1: Random . . . 11
Policy 2: Popularity . . . 11
9. Policy 3: Epsilon-Greedy . . . 12
10. Policy 4: LinUCB . . . 13
11. What one interactive run does . . . 14
12. Interface, controls, and every panel . . . 15
13. Metrics and how to interpret them . . . 16
14. Offline evaluation script . . . 17
15. One round from beginning to end . . . 19
16. Code walkthrough by function . . . 20
17. Assumptions, limits, and things not implemented . . . 21
18. Glossary, quick formulas, and references . . . 22

READING THE RESULTS Likes are sampled outcomes. Oracle probability and regret are computed from the simulator that generated those outcomes. They are not direct measurements of how real viewers respond.

1. Project purpose and mental model

Movie Bandit Lab demonstrates a recommendation problem as sequential decision-making. A policy repeatedly chooses a movie, receives a simulated response, and may adapt. The interface makes the decision process visible: it compares reward curves, summarizes the viewer profile, exposes likely movies, shows one decision at a time, and reports learned LinUCB telemetry.

The core bandit vocabulary

Term	Meaning in this project
Round / time step	One decision and one Bernoulli reward. A run contains 10 to 500 rounds per policy.
Context, x	The selected viewer represented by 19 normalized genre weights.
Action / arm, a	One movie position in the fixed 42-candidate slate. A policy chooses one arm per round.
Reward, r	A binary sampled outcome: 1 for a simulated like and 0 for no-like.
Policy / agent	The action-selection rule: Random, Popularity, Epsilon-Greedy, or LinUCB.
Oracle	The fixed probability-generating rule. It supplies p(like) for every viewer/movie pair.
Exploration	Trying actions to learn or reduce uncertainty.
Exploitation	Choosing an action favored by current estimates or fixed popularity.

THE IMPORTANT DISTINCTION

This is a simulation, not an online product connected to real viewers. The feedback probability is hand-specified and frozen. The policies learn from sampled rewards inside a run, but the simulator knows the generating probability so it can display oracle probability and calculate regret.

At a high level, each turn is: observe the selected viewer context; score or sample an action; sample feedback from the environment; update the selected policy; record reward and regret. The environment is stationary during a run: the selected viewer and the probability for each movie stay fixed.

2. Project files and how they fit together

The files separate data preparation, the simulated world, policy logic, user interface, and multi-seed evaluation. This makes it possible to inspect the math without reading Streamlit layout code, or to study the front end without changing the model.

File	Responsibility	What to inspect
environment.py	Loads MovieLens, builds splits, candidate slate, contexts, popularity, and the reward environment.	Feature construction, `candidate_movies`, `predict_probabilities`, `step`.
agents.py	Implements the four action-selection and update rules using NumPy.	State variables and the exact `select_action` / `update` logic.
app.py	Streamlit presentation, user controls, four-policy interactive run, charts, inspector, explanations.	`run_demo`, chart helpers, sidebar inputs, result/session-state logic.
run_evaluation.py	Runs an offline multi-seed experiment over real MovieLens training users; saves two plots.	Seed loop, user sequence, regret, confidence bands, aggregate output.
data/ml-100k/	Source MovieLens 100K files, including ratings, movie metadata, genres, user metadata, and split files.	The app directly reads `u.data` and `u.item`.
README.md	Installation, launch, and offline evaluation instructions.	Uses the project `.venv` and Streamlit command.
results_reward.png / results_regret.png	Saved evaluation plots from the offline script.	Stored outputs; they are not recalculated by opening the app.
verification_report.md	Audit record for the stored evaluation run and slate.	Historical run configuration and saved aggregate results.

Dependency roles

- NumPy supplies vectors, matrices, random generators, dot products, inverses, and Bernoulli sampling.
- Pandas reads and filters the ratings and movie tables, groups ratings by user/movie, and assembles display tables.
- Streamlit runs the interactive page, sidebar controls, session state, progress UI, and expanders.
- Plotly draws interactive cumulative reward/regret and genre-weight charts.
- Matplotlib draws the saved offline evaluation curves.
- streamlit-option-menu provides the policy focus selector; the selection changes the view, not which policies are run.

The project is launched with `.venv/bin/python -m streamlit run app.py`. The offline comparison is launched separately with `.venv/bin/python run_evaluation.py`. Opening the UI does not run the 5-seed offline experiment.

3. MovieLens data and preprocessing

The project uses the MovieLens 100K benchmark distributed by GroupLens. The official dataset page describes the collection as 100,000 ratings from 1,000 users across about 1,700 movies, released in April 1998. The local README and source tables are included under `data/ml-100k/`. The project uses the 1-to-5 rating values and timestamps from `u.data`, and genre flags and titles from `u.item`.

Input files and columns used

```
u.data: user_id | item_id | rating | timestamp
u.item: item_id | title | release_date | video_release_date | imdb_url | 19 genre flags
```

The movie genre flags have a fixed order. The same order is used for movie vectors and viewer vectors, so a feature index has the same meaning in both.

Feature indices	Genre names
0-4	unknown, Action, Adventure, Animation, Children's
5-9	Comedy, Crime, Documentary, Drama, Fantasy
10-14	Film-Noir, Horror, Musical, Mystery, Romance
15-18	Sci-Fi, Thriller, War, Western

Per-user chronological split

All ratings are stably sorted by user ID and timestamp. For each user independently, the first floor(0.8 times that user's rating count) rows become training data; the remaining later rows become test data. The oracle, candidate selection, user taste vectors, and MovieLens popularity signals use training data only. The interactive demo does not query the test partition. The split is temporal within each user, not a random rating-level split.

WHY THIS MATTERS

Keeping later ratings out of the profiles and the MovieLens popularity computation avoids using future ratings as input to the demo. It does not turn the simulator into a held-out recommendation evaluation: the user interface still samples feedback from a hand-built oracle.

MovieLens limits to state clearly

The dataset is sparse: each user rated only a small fraction of all movies. The action slate intentionally narrows the catalog to 30 sufficiently observed titles. The dataset release is old and its genre labels are coarse. It is suitable for an educational offline demonstration, not a claim about present-day film preferences.

Official source: GroupLens MovieLens 100K dataset page. Review the included dataset README for its usage terms.

4. Viewer context and the 11 profiles

A real MovieLens viewer is encoded as a 19-dimensional genre profile. For every training rating of 4 or 5, the code adds the movie's binary genre flags to a genre-count vector. It then divides by the total count of genre mentions. A multi-genre movie adds one count to each of its genres, so the vector normalizes genre mentions, not the number of liked movies.

```
c_g(u) = sum of genre flag g over training movies rated 4 or 5 by user u
x_g(u) = c_g(u) / sum_j c_j(u)
If sum_j c_j(u) = 0, the profile remains all zeros.
```

A viewer profile can contain many non-zero genres. A value such as 0.20 means that genre accounts for 20 percent of the normalized positive-rating genre mentions. The vector is an input feature, not a probability distribution over movies.

How the playful profile names map to users

The ten genre-labeled profiles are not manually edited personas. The code builds a one- or two-genre target vector, ranks real training users by dot product between their profile and that target, then takes the highest-ranked not-yet-used user. Processing the labels in order produces distinct users when possible. The label is a convenient presentation name; it is not an identity or demographic statement.

Profile label	Alignment target	Context source	Highest context genres
Action junkie	Action	Real MovieLens training user #578	Action, Thriller, Adventure
Rom-com fan	Romance, Comedy	Real MovieLens training user #726	Comedy, Drama, Romance
Sci-fi dreamer	Sci-Fi	Real MovieLens training user #202	Comedy, Sci-Fi, Action
Drama devotee	Drama	Real MovieLens training user #656	Drama, Comedy
Comedy fan	Comedy	Real MovieLens training user #208	Comedy, Romance, Drama
Mystery sleuth	Mystery	Real MovieLens training user #626	Drama, Mystery, Crime
Horror buff	Horror	Real MovieLens training user #366	Horror, Thriller, Action
Adventure seeker	Adventure	Real MovieLens training user #337	Adventure, Action, Sci-Fi
Animation fan	Animation	Real MovieLens training user #822	Comedy, Animation, Children's
War-story watcher	War	Real MovieLens training user #212	Drama, War, Romance
Bollywood buff	All Bollywood item genre flags	Synthetic; mean of Bollywood demo genres	Drama, Comedy, Romance

The synthetic Bollywood buff context is the sum of the 12 Bollywood genre vectors normalized to total one. This gives the demo a consistent synthetic viewer that can overlap with the hand-added titles. It is explicitly separate from the real MovieLens users used by the offline evaluation.

Genre names are categorical metadata; the project does not use age, gender, occupation, user IDs as model features, collaborative filtering, embeddings, or natural-language movie descriptions in the policy context.

5. Action space: exactly 42 movie arms

An arm is a candidate movie. The policy receives an integer action index from 0 through 41; the environment maps that index through `candidate_movies` to the MovieLens item ID or negative demo ID. The negative IDs cannot collide with positive MovieLens item IDs.

Selection of the 30 MovieLens titles

The pipeline counts training ratings per movie, keeps MovieLens item IDs with at least 50 training ratings, and sorts eligible items by descending training count and then ascending ID. It repeatedly chooses the remaining movie with the greatest genre coverage gain. When two candidates add the same number of previously uncovered genres, training count wins; an ID tie is broken by lower ID. The routine continues until 30 titles are selected, then freezes that MovieLens slate.

```
score(item) = (new genre flags covered, training rating count, -item_id)
choose the lexicographic maximum at each greedy step
```

This procedure balances coverage and observation volume. It is a heuristic, not a global optimization of genre diversity or recommendation quality.

Twelve hand-added Bollywood demonstration titles

Demo ID	Title	Illustrative genre flags	Popularity
-1	3 Idiots (2009)	Comedy, Drama, Romance	0.98
-2	Dangal (2016)	Drama	0.96
-3	Sholay (1975)	Action, Adventure, Comedy, Drama	0.94
-4	Dil Chahta Hai (2001)	Comedy, Drama, Romance	0.88
-5	Gully Boy (2019)	Drama, Musical	0.84
-6	Queen (2013)	Comedy, Drama, Romance	0.86
-7	Andhadhun (2018)	Comedy, Crime, Thriller	0.85
-8	Lagaan (2001)	Drama, Musical, Romance	0.91
-9	PK (2014)	Comedy, Drama, Sci-Fi	0.95
-10	Zindagi Na Milegi Dobara (2011)	Adventure, Comedy, Drama	0.90
-11	Chak De! India (2007)	Drama	0.89
-12	Barfi! (2012)	Comedy, Drama, Romance	0.87

These titles have no MovieLens ratings. Their genre vectors and popularity values were hand-assigned to place Bollywood films in the same 19-feature action representation. They are included in the interactive action slate and the offline script's candidate set. The synthetic Bollywood viewer is used only by the interactive profile menu; offline evaluation samples real training users.

5A. Complete ordered candidate slate

This is the action-index order used by the app and both policy loops. The arm index is the array position; the item ID is the underlying movie identifier.

Arm	Item ID	Movie title	Source
00	172	Empire Strikes Back, The (1980)	MovieLens 100K
01	302	L.A. Confidential (1997)	MovieLens 100K
02	95	Aladdin (1992)	MovieLens 100K
03	288	Scream (1996)	MovieLens 100K
04	423	E.T. the Extra-Terrestrial (1982)	MovieLens 100K
05	97	Dances with Wolves (1990)	MovieLens 100K
06	48	Hoop Dreams (1994)	MovieLens 100K
07	50	Star Wars (1977)	MovieLens 100K
08	100	Fargo (1996)	MovieLens 100K
09	258	Contact (1997)	MovieLens 100K
10	181	Return of the Jedi (1983)	MovieLens 100K
11	286	English Patient, The (1996)	MovieLens 100K
12	294	Liar Liar (1997)	MovieLens 100K
13	300	Air Force One (1997)	MovieLens 100K
14	1	Toy Story (1995)	MovieLens 100K
15	121	Independence Day (ID4) (1996)	MovieLens 100K
16	174	Raiders of the Lost Ark (1981)	MovieLens 100K
17	127	Godfather, The (1972)	MovieLens 100K
18	7	Twelve Monkeys (1995)	MovieLens 100K
19	56	Pulp Fiction (1994)	MovieLens 100K
20	98	Silence of the Lambs, The (1991)	MovieLens 100K
21	237	Jerry Maguire (1996)	MovieLens 100K
22	117	Rock, The (1996)	MovieLens 100K
23	222	Star Trek: First Contact (1996)	MovieLens 100K
24	204	Back to the Future (1985)	MovieLens 100K
25	173	Princess Bride, The (1987)	MovieLens 100K
26	69	Forrest Gump (1994)	MovieLens 100K
27	79	Fugitive, The (1993)	MovieLens 100K
28	405	Mission: Impossible (1996)	MovieLens 100K
29	210	Indiana Jones and the Last Crusade (1989)	MovieLens 100K
30	-1	3 Idiots (2009)	Bollywood demo
31	-2	Dangal (2016)	Bollywood demo
32	-3	Sholay (1975)	Bollywood demo
33	-4	Dil Chahta Hai (2001)	Bollywood demo
34	-5	Gully Boy (2019)	Bollywood demo
35	-6	Queen (2013)	Bollywood demo
36	-7	Andhadhun (2018)	Bollywood demo
37	-8	Lagaan (2001)	Bollywood demo
38	-9	PK (2014)	Bollywood demo
39	-10	Zindagi Na Milegi Dobara (2011)	Bollywood demo
40	-11	Chak De! India (2007)	Bollywood demo
41	-12	Barfi! (2012)	Bollywood demo

6. Movie features and popularity

Each MovieLens movie is represented by a 19-position multi-hot genre vector g_a. A flag is 1 if the movie has that genre and 0 otherwise. A title can have several genres. The hand-added Bollywood items are mapped into the same feature order using their manually assigned genre names.

Movie popularity has two related but distinct uses

- Environment popularity feature: For the original 30 MovieLens candidates, count training ratings of 4 or 5 per movie, then min-max scale the counts across those 30 titles: s_a = (count_a - min_count) / (max_count - min_count). If the minimum equals the maximum, the code uses zero scores. For Bollywood items, use manually assigned values from 0 to 1.
- Popularity policy estimate: In the app, compute each candidate's mean positive-rating indicator from training ratings. If a title has no training observations, fall back to the fixed popularity score. The PopularityPolicy always selects the largest of these values.
- Oracle use: The frozen environment adds the movie popularity score with coefficient 1.0 when calculating p(like).

NOT THE SAME SIGNAL

MovieLens movie_popularity_score uses min-max-scaled counts of high ratings among the original 30 candidates. The PopularityPolicy uses per-candidate high-rating rates with the popularity score as fallback. Both are derived from training data for MovieLens titles, but they are not identical quantities.

No demographic features, movie descriptions, user-user similarity, movie-movie similarity, rating matrix factorization, or learned text embeddings enter the model in this project.

7. The environment: probability and reward math

For a viewer context x and candidate movie a, calculate genre overlap as a dot product between the viewer's normalized positive-genre profile and the movie's binary genre vector:

```
overlap(u, a) = x(u)^T g(a) = sum over 19 genres of x_j(u) * g_j(a)
logit(u, a) = 4.0 * overlap(u, a) + 1.0 * popularity(a) - 1.5
p(u, a) = sigmoid(logit(u, a)) = 1 / (1 + exp(-logit(u, a)))
```

The logit is a real-valued score. The sigmoid converts it to a number strictly between 0 and 1. A positive genre match increases probability; larger fixed popularity also increases it; the negative intercept lowers the baseline. The implementation clips the logit to [-700, 700] before evaluating the exponential for numerical safety.

Worked example (illustrative values)

```
Suppose genre overlap = 0.40 and popularity = 0.75.
logit = 4.0 * 0.40 + 1.0 * 0.75 - 1.5 = 0.85
p(like) = 1 / (1 + exp(-0.85)) = about 0.701, or 70.1%.
```

The app samples a Bernoulli random variable with this probability: r ~ Bernoulli(p). The environment does not return the probability as the reward; it returns one binary draw. A 70.1% p(like) means repeated independent choices under that same pair have an expected like rate near 70.1%, not that this one choice must be a like.

A fixed oracle, a changing learner

The probability formula and features do not change as policies learn. What changes is the policy state and action. Epsilon-Greedy updates a sample average for chosen movies. LinUCB updates a separate linear model per chosen movie. Random and Popularity do not learn from sampled reward. The environment itself resets to the same viewer context each turn and uses its seeded RNG to draw feedback.

ORACLE DOES NOT MEAN PERFECT REAL-WORLD TRUTH

Oracle means the simulator knows the probability rule used to generate outcomes. It is a ground-truth benchmark only inside this constructed environment. It is not a fitted model of human responses.

8. Policy 1: Random

Random samples a uniformly distributed candidate index on every turn. With 42 actions, each arm has probability 1/42 of being selected on each independent selection.

```
a_t = draw an integer uniformly from {0, 1, ..., 41}
update(a_t, x_t, r_t): no operation
```

It consumes random state but not feedback history. The context argument is unused. The policy is therefore a blind-exploration baseline. Over T rounds, the expected number of times a particular movie is selected is T/42, though the realized count varies.

Policy 2: Popularity

Popularity deterministically selects the movie whose training-based estimate is largest:

```
a_t = argmax_a train_like_rate(a)
```

The policy ignores viewer context and reward observations inside a run. Since the estimates are fixed, it normally selects the same movie every round. `numpy.argmax` returns the first index when values tie, so order matters under a tie. Bollywood candidates have no train ratings and use their fixed popularity value as fallback in the UI/evaluation constructors.

How to interpret the baselines

Random asks: how much reward do we get without using information? Popularity asks: how well does a fixed broadly popular movie do? Neither is personalized nor adapts online. The Popularity baseline may beat a learning policy over short horizons because it starts with prior data and does not spend rounds learning; that is not evidence that exploration is never useful.

9. Policy 3: Epsilon-Greedy

Epsilon-Greedy estimates one average reward per movie and ignores viewer context. It balances exploration and exploitation with epsilon = 0.1.

```
with probability epsilon = 0.1: choose a uniformly random movie
with probability 1 - epsilon = 0.9: choose argmax_a Q_hat(a)
```

In code, the random exploration branch is drawn first by checking whether a uniform random value is less than epsilon. If so, the movie index is sampled uniformly. Otherwise, select the first movie with the greatest current estimated value.

Sample-average update

```
N_a <- N_a + 1
Q_hat(a) <- Q_hat(a) + (r - Q_hat(a)) / N_a
```

This is algebraically equivalent to the arithmetic mean of all observed rewards for arm a. It stores `counts[a]` and `values[a]`; it updates only the chosen arm. Because rewards are 0 or 1, the estimate is the empirical like fraction for that movie within this run. It is not the MovieLens training rate and it is not conditioned on the current viewer.

Initialization and display labels

All values and counts start at zero. On the first exploitation round, all actions tie and `argmax` chooses index 0. The app marks exploration using `last_exploring`, which records whether the epsilon branch was used for that particular decision. It marks exploitation otherwise. A random exploratory action can happen to choose the current best movie; its behavior label still correctly reflects the branch used.

A 10% exploration probability is a fixed setting, not a learned schedule. It does not shrink with experience. At 500 rounds, the expected number of exploration selections is 50, but the actual number can vary.

10. Policy 4: LinUCB

LinUCB is the contextual policy. It keeps one linear model per movie, using the selected viewer's 19-dimensional genre vector. It chooses a movie by adding a confidence bonus to the predicted mean reward.

State for each action a

```
d = 19 context features
A_a starts as the d-by-d identity matrix I_d
b_a starts as the all-zero d-vector
theta_a = inverse(A_a) * b_a
```

A_a summarizes context observations for movie a; b_a summarizes reward-weighted contexts. The identity initialization is ridge-style regularization and makes A invertible at initialization. The learner uses matrix inversion directly with NumPy.

Action score

```
mean_a(x) = theta_a^T x
uncertainty_a(x) = sqrt(max(0, x^T inverse(A_a) x))
UCB_a(x) = mean_a(x) + alpha * uncertainty_a(x)
a_t = argmax_a UCB_a(x_t)
alpha = 1.0 in this project
```

The first term rewards actions predicted to perform well. The second rewards actions whose estimate is uncertain for this context. With A initially I and b initially zero, the initial mean is 0 and initial uncertainty is sqrt(x^T x). Since all users have a fixed context during an interactive run, repeated selection of one movie makes that movie's uncertainty decrease for that context.

Update after the chosen outcome

```
A_a <- A_a + x * x^T
b_a <- b_a + r * x
```

Only the chosen movie's matrices are updated. The score model is linear in x and the observed reward is binary. The prediction is not passed through a sigmoid and is not guaranteed to stay between 0 and 1. The environment probability, by contrast, is logistic. This is a deliberate algorithm comparison, not a matched model specification.

Exploration label and explanation panel

To label a LinUCB decision, the app also finds the arm with the largest mean alone. If LinUCB's selected action differs, the decision is called exploration; otherwise exploitation. This is an explanation convention for the demo. The bonus is considered in every LinUCB score, even on turns labelled exploitation.

After the run, the LinUCB panel recomputes all scores using its final learned state and the selected viewer context. It highlights the current highest UCB action and shows the learned coefficients for that movie. That post-run action is not necessarily the movie chosen on the last round, and the learned coefficients are not oracle weights.

11. What one interactive run does

The sidebar provides a viewer, focus policy, round horizon, and Run button. A run evaluates all four policies, regardless of which focus policy is selected. Focus policy only controls which trace and single-policy panels are emphasized.

Stage	What the app does
1. Construct independent state	Make one environment and one policy instance for each of the four policy names. The environment/policy random seeds are the run seed plus that policy's index.
2. Freeze oracle vector	Ask the Random environment for the selected user's 42 probabilities. Since the formula is deterministic, the same vector applies to all four policies.
3. Iterate turns	For each round and then each policy: reset that environment to the viewer, compute LinUCB diagnostics if applicable, select one arm, sample reward, label behavior, and append the decision record.
4. Update the learner	Call the policy update method. Random and Popularity no-op; Epsilon-Greedy updates one average; LinUCB updates one arm's matrices.
5. Accumulate measures	Add reward to total/cumulative likes. Add p_best - p_chosen to regret. Store movie ID/title, oracle probability, reward, behavior, and LinUCB score details.
6. Render results	Draw charts, update metrics and scorecard, and expose the selected policy's inspector record.

Seed and reproducibility

The interactive button calls `run_demo(..., seed=17, ...)`. Each policy and its environment receive a deterministic seed offset. Re-running the same viewer and horizon from freshly initialized policy state should reproduce the same sampled trace under the same software and data versions. The selected policy is not part of the experiment key; changing it reuses saved results. The result key is `(selected_profile, number_of_rounds)`.

Random streams are policy-specific, not paired common-random-number draws. Since action probabilities differ and each environment has its own generator, a Random policy and LinUCB do not receive the exact same reward outcome sequence. The interactive comparison is a clear demonstration, but not a controlled paired statistical study.

Session state and inspector

The run result is stored in `st.session_state["demo_result"]` with the viewer/horizon key. When the viewer or horizon changes, `has_matching_result` becomes false and result-dependent displays request a fresh run. Switching policy does not invalidate the result. The inspector slider key includes profile, focus policy, and horizon so it tracks the currently displayed trace. Early and late buttons use callbacks to set the slider state before the Streamlit rerun.

12. Interface, controls, and every panel

Selectable experiment dimensions

Dimension	Options	Effect
Simulated viewer	11 profiles	Selects the fixed context vector used by the oracle and LinUCB.
Focus policy	4 policies	Highlights one policy and chooses which trace appears in selected-policy metrics and the inspector. All four policies still run.
Rounds per policy	10, 20, 30, ... 500 (50 values)	Sets the learning horizon for each of the four policies.
Run button	Pressed / not pressed	Starts a run for the current viewer and horizon; values are stored for subsequent view changes.

There are 11 x 4 x 50 = 2,200 viewer/focus-policy/horizon combinations. The Run button is an action, not an additional algorithm configuration. All policies receive the same horizon.

Panel-by-panel

Panel	What it means
Movie slate	42 candidate actions; tooltip notes 12 are hand-added demo movies.
Liked genres	Count of non-zero entries in the current viewer context; not a count of all genre mentions.
Likes - selected policy	Sum of observed Bernoulli rewards for the focused policy.
Average regret	Mean per-round difference between the best oracle probability for this viewer and the probability of each chosen movie.
Learning trajectory	All four cumulative reward or cumulative regret curves; focus policy is made more prominent. Hover compares policies at the same round; a vertical guide follows the inspector round.
Viewer taste	Horizontal genre profile chart, showing up to 12 non-zero genres.
Most likely to land	Top five fixed oracle probabilities before the run; includes genre overlap and popularity.
What LinUCB learned	Learned genre coefficients, current final-state UCB choice, mean estimate, uncertainty bonus, and oracle construction breakdown.
Round-by-round inspector	Chosen title, round, exploration/exploitation label, oracle p(like), reward, and LinUCB equation when selected.
Policy scorecard	Each policy's total likes, average regret, exploration rate, and highest total reward badge.
How it works / action slate	Expandable text explanation and all 42 actions with current viewer-specific oracle values.

13. Metrics and how to interpret them

Cumulative reward

```
R_n = sum from t=1 to n of r_t
```

Because each r_t is 0 or 1, cumulative reward is the running count of likes. It is stochastic. A policy may have a good expected probability and still receive an unlucky sequence of no-likes.

Cumulative and average regret

```
p*_u = max over all 42 actions a of p(u, a)
regret_t = p*_u - p(u, a_t)
cumulative_regret_n = sum from t=1 to n of regret_t
average_regret = cumulative_regret_T / T
```

This is oracle probability regret, sometimes called pseudo-regret. It compares expected probabilities, not the sampled r_t. As long as the action belongs to the same candidate set, each per-round gap is nonnegative. A low regret does not necessarily equal more realized likes in one short sample.

Exploration rate

The leaderboard computes the percentage of recorded decisions whose behavior label is Exploration. Random is always labelled exploration, Popularity always exploitation, Epsilon-Greedy follows its epsilon branch, and LinUCB uses the mean-only comparison rule. Therefore, the columns use a shared display concept but policy-specific classification rules.

Potential interpretation traps

- A larger total reward in one fixed-seed run is not by itself evidence of statistical superiority.
- The simulator oracle already combines genre match and popularity. The app's Popularity policy also has a training-data estimate; these should not be conflated.
- LinUCB predicts a linear reward estimate and adds uncertainty; its learned estimate is not the oracle probability.
- The top-five oracle list is computed before the run and does not reflect policy learning.
- The final LinUCB spotlight is recomputed after learning and can differ from the selected round's decision.

14. Offline evaluation script

`run_evaluation.py` is separate from the UI run. It compares the policies over five seeds and 2,000 rounds per policy per seed, using real MovieLens training users sampled with replacement. At each seed it generates one 2,000-user sequence and supplies that same sequence to each policy. Each policy gets a fresh state. Each policy environment uses the seed plus 1,000 for its reward RNG.

The evaluation caches the frozen oracle probability matrix for every real training user and candidate movie before the loops. It also samples 50 diagnostic users using a separate generator seeded 42, reporting the number of distinct best-probability movies and mean best-versus-median probability gap. These are diagnostics of the constructed oracle, not performance results.

Outputs and aggregation

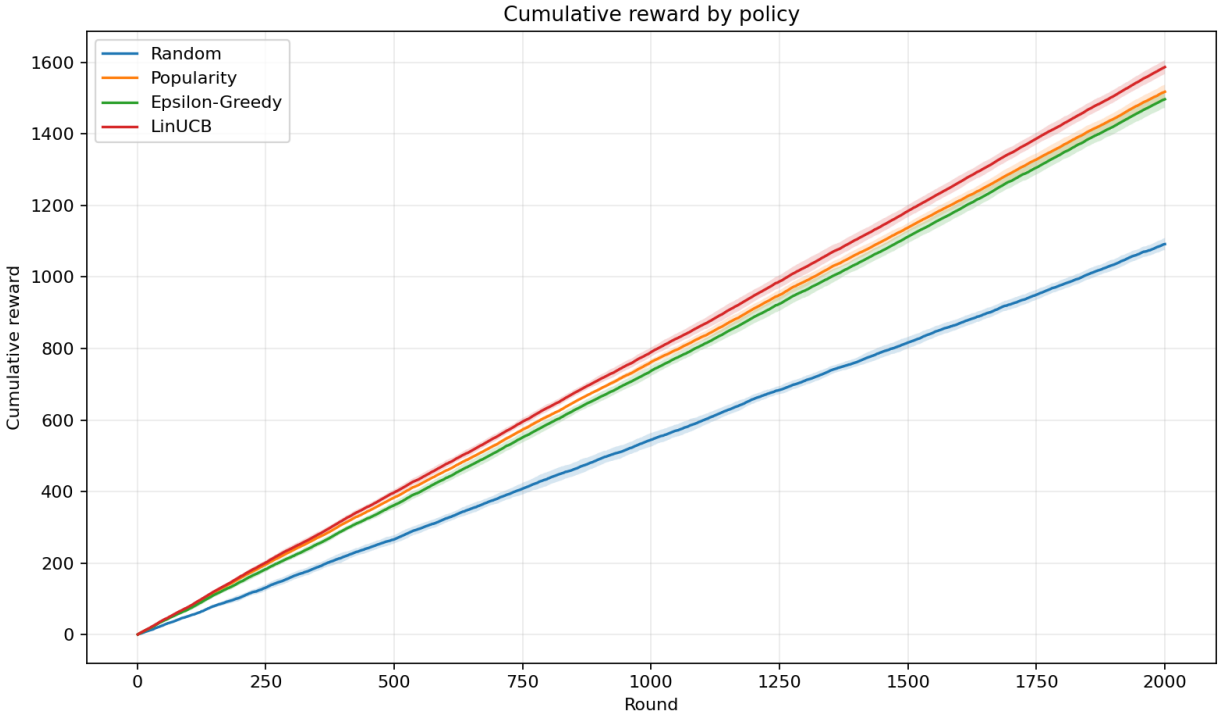
- For each seed and policy, it records a cumulative reward curve, cumulative pseudo-regret curve, total reward, and number of Bollywood actions.
- For each plot, it averages the five seed curves and shades plus/minus one population standard deviation.
- It reports mean and standard deviation of final reward totals across the five seeds.
- It writes `results_reward.png` and `results_regret.png` in the project root. Running the script overwrites these plot files.
- The synthetic Bollywood buff is not part of offline evaluation; users are sampled from real MovieLens training user contexts.

Stored evaluation artifacts

The checked-in `verification_report.md` records a five-seed, 2,000-round run. It reports mean total rewards: Random 1,092.0 (standard deviation 16.8), Popularity 1,517.8 (19.5), Epsilon-Greedy 1,496.8 (23.7), LinUCB 1,586.8 (19.4). It reports Bollywood actions across the five seeds: Random 2,890; Popularity 10,000; Epsilon-Greedy 3,917; LinUCB 0. Treat these as stored run results tied to that exact code/configuration, not a guarantee about future runs.

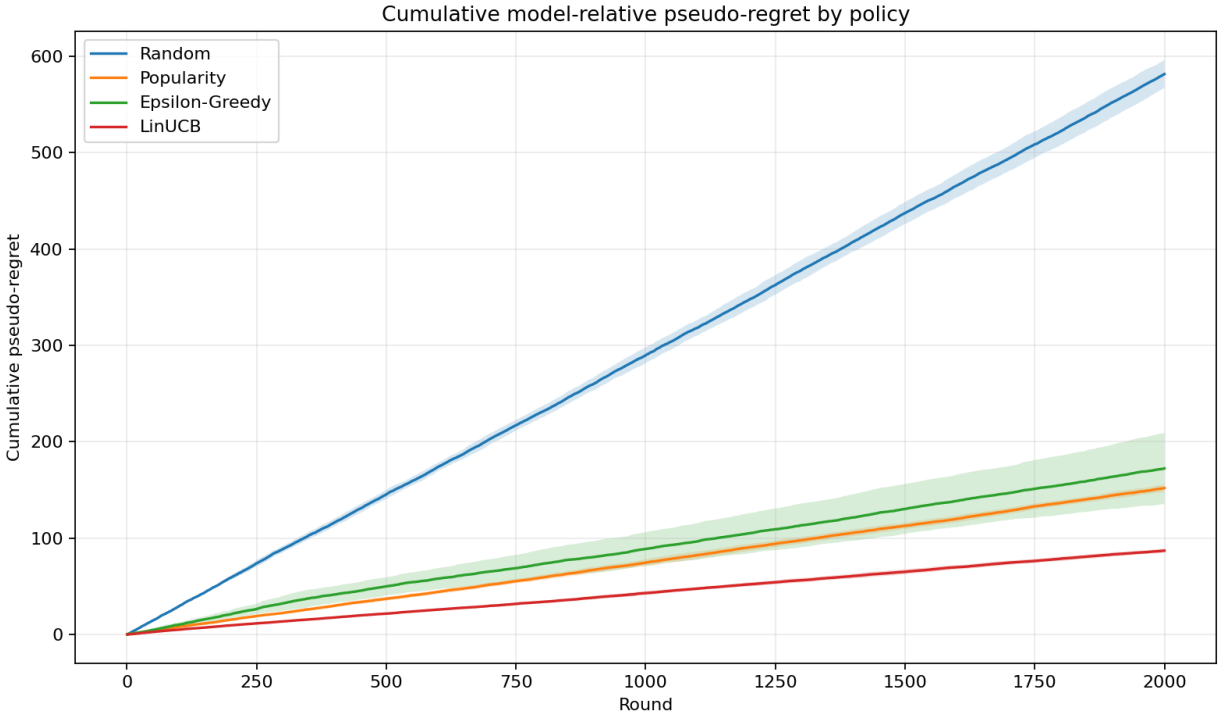
IMPORTANT LIMIT This evaluation still samples rewards from the hand-specified oracle. It is a multi-seed evaluation of algorithms inside the simulator, not a train/test evaluation against held-out observed ratings or an online A/B test with users.

Stored cumulative reward comparison



Stored project plot. The plotted statistic is averaged over the five seeds described above; it is not recreated while generating this guide.

Stored cumulative pseudo-regret comparison



Stored project plot. The plotted statistic is averaged over the five seeds described above; it is not recreated while generating this guide.

15. One round from beginning to end

The following pseudocode compresses the interactive `run_demo` routine without hiding the policy-specific updates:

```
for round t in 1..T:
    for policy in [Random, Popularity, Epsilon-Greedy, LinUCB]:
        x = environment.reset(selected_viewer)
        if policy is LinUCB:
            compute mean, uncertainty, UCB for each of 42 arms
            remember mean-only greedy arm for behavior label
        a = policy.select_action(x)
        p = frozen_oracle_probability[selected_viewer, a]
        r = draw Bernoulli(p)
        behavior = policy-specific explore/exploit label
        save arm, title, p, r, behavior, optional UCB telemetry
        policy.update(a, x, r)
        reward_total += r
        regret_total += best_probability - p
```

Illustrative numeric event

Suppose the current viewer's best oracle probability is 0.84 and the chosen action has oracle probability 0.70. The probability regret for this decision is 0.14. If the environment samples reward 1, the observed reward is a like; if it samples 0, it is a no-like. In both cases regret remains 0.14 because regret is computed from the probabilities, not the outcome.

For Epsilon-Greedy, if this action has previously been chosen 4 times with average 0.50 and it now receives reward 1, the new value is 0.50 + (1 - 0.50)/5 = 0.60. For LinUCB, the same reward updates only the chosen movie: A_a gains x x^T and b_a gains x. Its next score may change because both its mean and uncertainty can change.

The sequence is intentionally easy to inspect: the `decisions` result stores one dictionary per policy per round, so the inspector can display the actual chosen action and its probabilities without rerunning the learning process.

16. Code walkthrough by function

Environment functions and data objects

Symbol	Role
GENRE_NAMES / MOVIE_COLUMNS	Canonical feature order and MovieLens item table schema.
candidate_movies / movie_titles	Ordered 42-action mapping and display title lookup.
movie_genre_vectors	MovieLens genre flags plus hand-built Bollywood vectors.
user_contexts / demo_user_contexts	Normalized real-user contexts and the synthetic Bollywood profile.
movie_popularity_score	Fixed movie popularity values used by the oracle.
MovieBanditEnvironment.reset(user_id)	Selects a viewer and returns a copy of that viewer's 19-feature context.
predict_probabilities(user_id, item_ids=None)	Applies the frozen logistic equation to one or all candidates.
step(action)	Validates a 0-41 integer action, maps it to an item ID, samples one Bernoulli reward.

App functions

Function	Role
build_profile_users	Maps ten genre labels to distinct real-user contexts and adds the synthetic profile.
make_policy	Constructs a fresh named policy, with epsilon 0.1 and LinUCB alpha 1.0.
linucb_scores	Calculates the per-arm LinUCB mean, bonus, and sum for display.
run_demo	Runs all policies, records each decision, reward totals, cumulative reward, regret, and learned LinUCB state.
make_reward_chart	Shows reward or regret learning curves, focus highlighting, unified hover, and inspector guide.
make_genre_chart	Displays non-zero viewer context genres or learned LinUCB coefficients.
oracle_breakdown	Separates genre term, popularity term, intercept, and oracle probability for the explanation panel.
render_explanation / render_how_it_works	Renders model telemetry and expandable concept explanations.

17. Assumptions, limits, and things not implemented

- Simulated response: no new human rating is collected. Likes are Bernoulli samples from a chosen logistic formula.
- Hand-set coefficients: genre weight 4, popularity weight 1, and intercept -1.5 are design choices, not fitted parameters.
- Popularity is partly constructed: MovieLens values use the project training split; Bollywood values are illustrative manual assignments.
- Fixed single viewer in the interactive run: a run does not rotate viewers. It repeatedly asks the same selected context.
- Policies are intentionally simple: no contextual neural model, collaborative filter, Thompson sampling, slate optimization, or delayed feedback.
- Independent app policies: each sees its own environment stream; random outcomes are not paired across strategies.
- Oracle regret is privileged: a real service would not know each user/action true probability.
- One-seed UI: the interactive page is a demonstration, while robust comparisons require the separate five-seed script or additional repetitions.
- LinUCB model mismatch: it estimates reward linearly while the simulator generates reward from a logistic rule.

The demo is most useful for explaining the mechanics of exploration, exploitation, contextual features, binary feedback, and uncertainty bonuses. Avoid presenting the UI as a production recommendation engine or interpreting one run as a real-world performance benchmark.

Suggested classroom walkthrough

```
1. Select a real genre-aligned profile and identify its 19-feature taste vector.
2. Inspect the top-five oracle recommendations before the run.
3. Select LinUCB, choose 120 rounds, and run all four policies.
4. Compare cumulative likes and then cumulative regret.
5. Inspect an early and a late LinUCB decision; explain mean + bonus.
6. Switch focus to Epsilon-Greedy and explain its per-movie average.
7. Select Bollywood buff and identify which inputs are synthetic.
```

18. Glossary, quick formulas, and references

Term	Plain-language definition
Arm	One movie option the policy can choose.
Bernoulli draw	A random 0/1 result with probability p of returning 1.
Contextual bandit	A sequence of action choices where each decision uses context and receives feedback only for the chosen action.
Dot product	Multiply corresponding vector entries and add the products; here it measures genre overlap.
Exploration bonus	An added score that favors uncertain actions.
Logit	The unbounded score before applying the sigmoid.
Oracle probability	The simulator's fixed p(like) for one viewer/movie pair.
Pseudo-regret	The gap in expected reward probability between oracle-best action and selected action.
Sigmoid	A function that maps a real number to a value between zero and one.
Training split	Earlier 80 percent of each user's chronologically sorted ratings.

Four equations to remember

```
Environment: p(u,a) = sigmoid(4 * x(u)^T g(a) + popularity(a) - 1.5)
Random: a ~ Uniform(42 actions)
Epsilon-Greedy: random with probability 0.1; otherwise argmax sample average
LinUCB: choose argmax_a [theta_a^T x + alpha * sqrt(x^T inverse(A_a) x)]
```

Project sources

- Local source of truth: `environment.py`, `agents.py`, `app.py`, `run_evaluation.py`, and `data/ml-100k/README`.
- Official dataset reference: https://grouplens.org/datasets/movielens/100k/.
- Historical stored experiment: `verification_report.md`, generated 21 October 2026; read its configuration before quoting any result.

This guide explains the current workspace implementation as inspected on 27 October 2026. It describes the current source behavior; if the code changes, source files remain authoritative.

FINAL TAKEAWAY

The project compares four policies over one fixed 42-movie action space. Context describes viewer taste, the environment turns context/action match into a probability, and sampled likes provide feedback. Random and Popularity are non-learning baselines; Epsilon-Greedy learns action averages; LinUCB learns context-dependent action estimates and explores where it is uncertain.